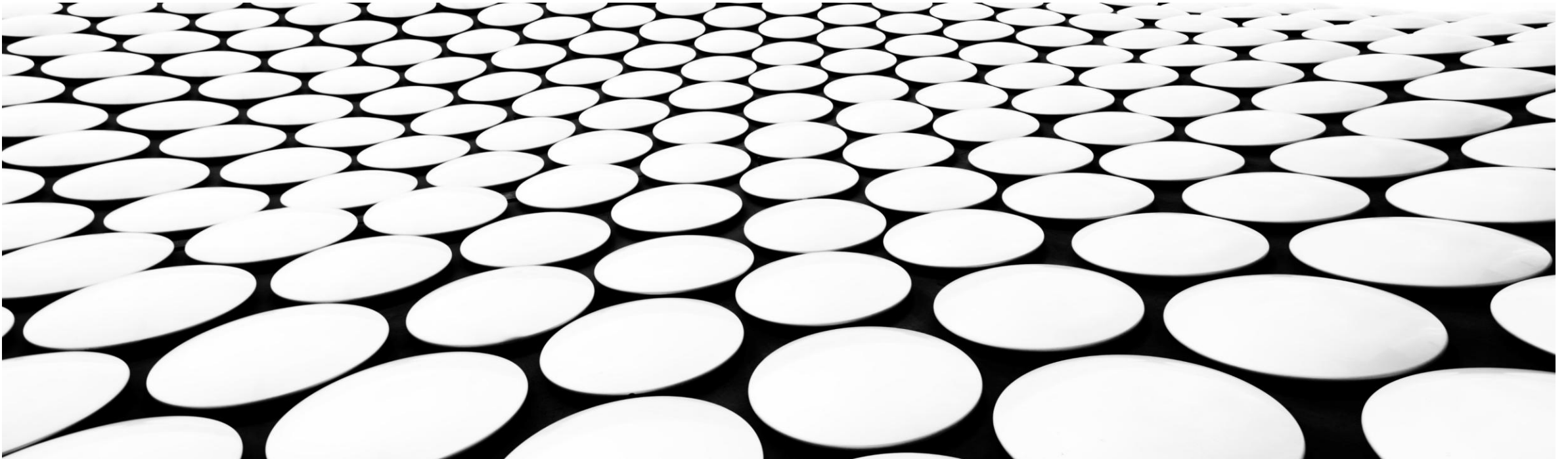
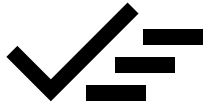


DATA INTEGRATION

Databases, data warehousing, ETL/ELT





DATA INTEGRATION

- Case study (Database review)
- Data Integration – ETL/ELT
- Transformation
 - Cleaning
 - Normalizing
 - Mapping
 - Deriving

Case study

Choosing Databases

15 Types of Databases and When to Use Them

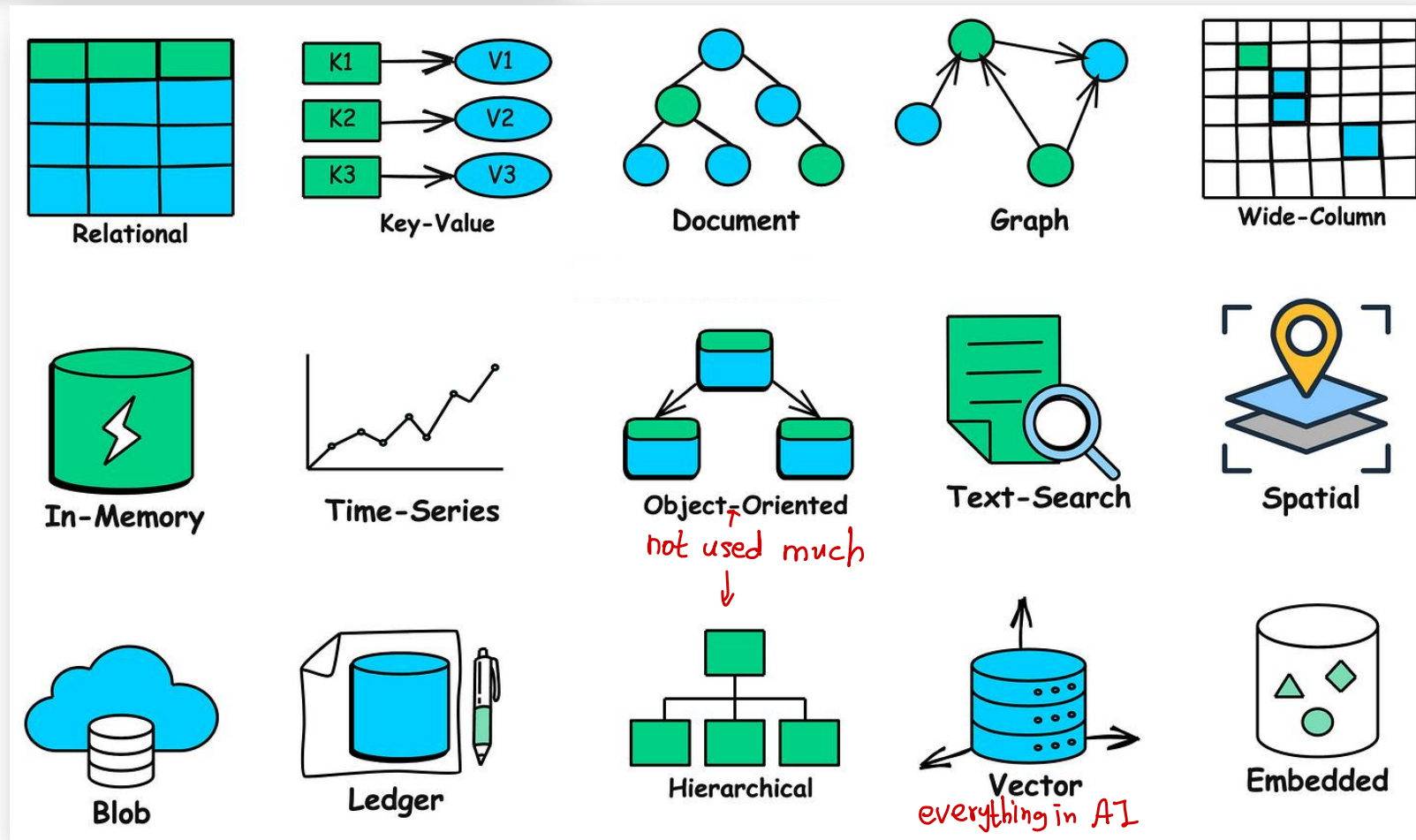
#4 System Design - Types of Databases



ASHISH PRATAP SINGH
MAR 24, 2024

Let's review...

Source



Purpose:

- Building a food delivery app similar to Uber Eats or DoorDash or other.

The system must support:

- (1) Orders and Payments
- (2) Menu and Restaurant Info
- (3) Search with phrases ("John* pizza")
- (4) Delivery tracking (real-time)
- (5) Location queries ("restaurant within 2 km")
- (6) Order History over time
- (7) Recommendations ("you may also like")
- (8) Caching of popular restaurants
- (9) Mobile App Local Storage
- (10) Audit/Payment Logs

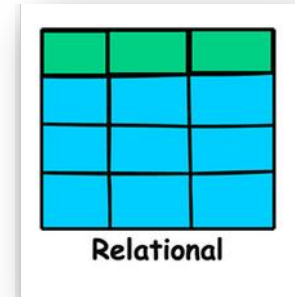
1. Orders & Payments

```
order = {
  "order_id": 1,
  "user_id": 10,
  "restaurant_id": 5,
  "total": 32.50,
  "status": "paid"
}
```

Requirements

- Strong consistency (no double charge)
- Transactions (all-or-nothing updates)
- Structured data (users, orders, payments)
- Referential integrity (valid user, valid order)

atomicity



Transactions

2. Menu and Restaurant Info

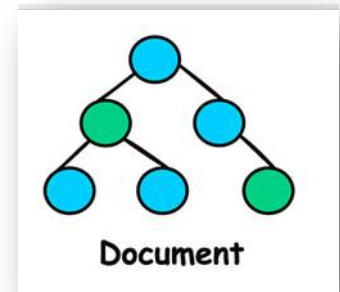
```
restaurant = {
  "name": "Pizza Place",
  "menu": [
    {"item": "Pepperoni", "price": 12},
    {"item": "Veggie", "price": 10}
  ]
}
```

nested items

Requirements

- Flexible schema (menus change often)
- Nested data (items, options, categories)
- Easy updates (add/remove items)

Flexible



3. Search (« John* pizza»)

🔧 Requirements

- Full-text search (not exact match)
- Ranking relevance (best results first)
- Tokenization (understand words, phrases)



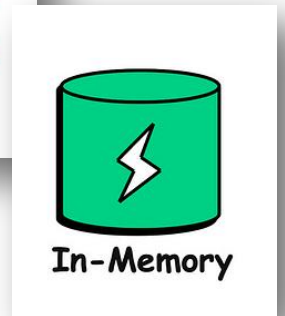
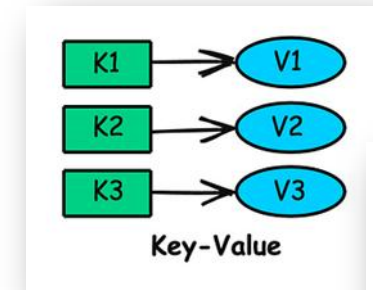
4. Delivery Tracking (Real-Time)

```
driver_location = {
  "driver_id": 7,
  "lat": 45.5017,
  "lon": -73.5673
}
```

→ Driver location

🔧 Requirements

- Very low latency reads/writes
- Frequent updates (every few seconds)
- Temporary data (latest position only)



Caching of information
Quickly accessible

5. Location Queries (« restaurants within 2km »)

🔧 Requirements

- Distance calculations
- Geographic indexing
- Spatial queries (radius, nearest neighbor)

often costly



Spatial

6. Order History over Time

```
{
  "timestamp": "2026-04-06 18:00",
  "orders_count": 120
}
```

🔧 Requirements

- Time-stamped data
- Trend analysis (orders per hour/day)
- Efficient aggregation over time windows

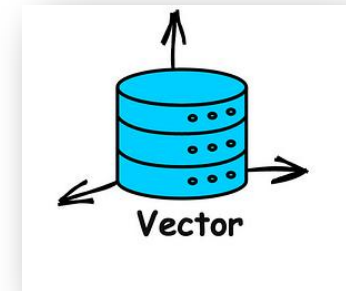


Time-based data

7. Recommendations (« you may also like »)

🔧 Requirements

- Similarity search (users/items)
- High-dimensional data (embeddings)
- Approximate nearest neighbor queries

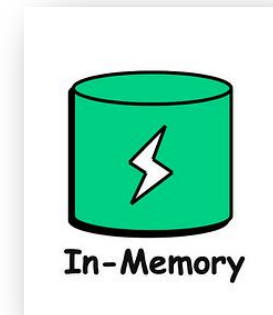


Embeddings

8. Caching Popular Restaurants

🔧 Requirements

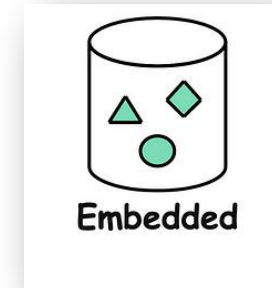
- Extremely fast reads
- Frequently accessed data
- Temporary / disposable storage



9. Mobile App Local Storage

Requirements

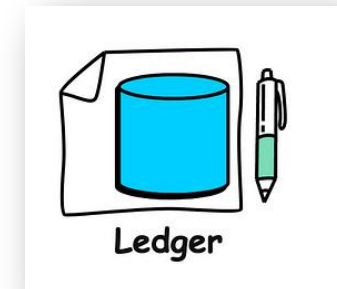
- Runs locally (no server)
- Lightweight
- Works offline
- Simple queries



10. Audit / Payment Logs

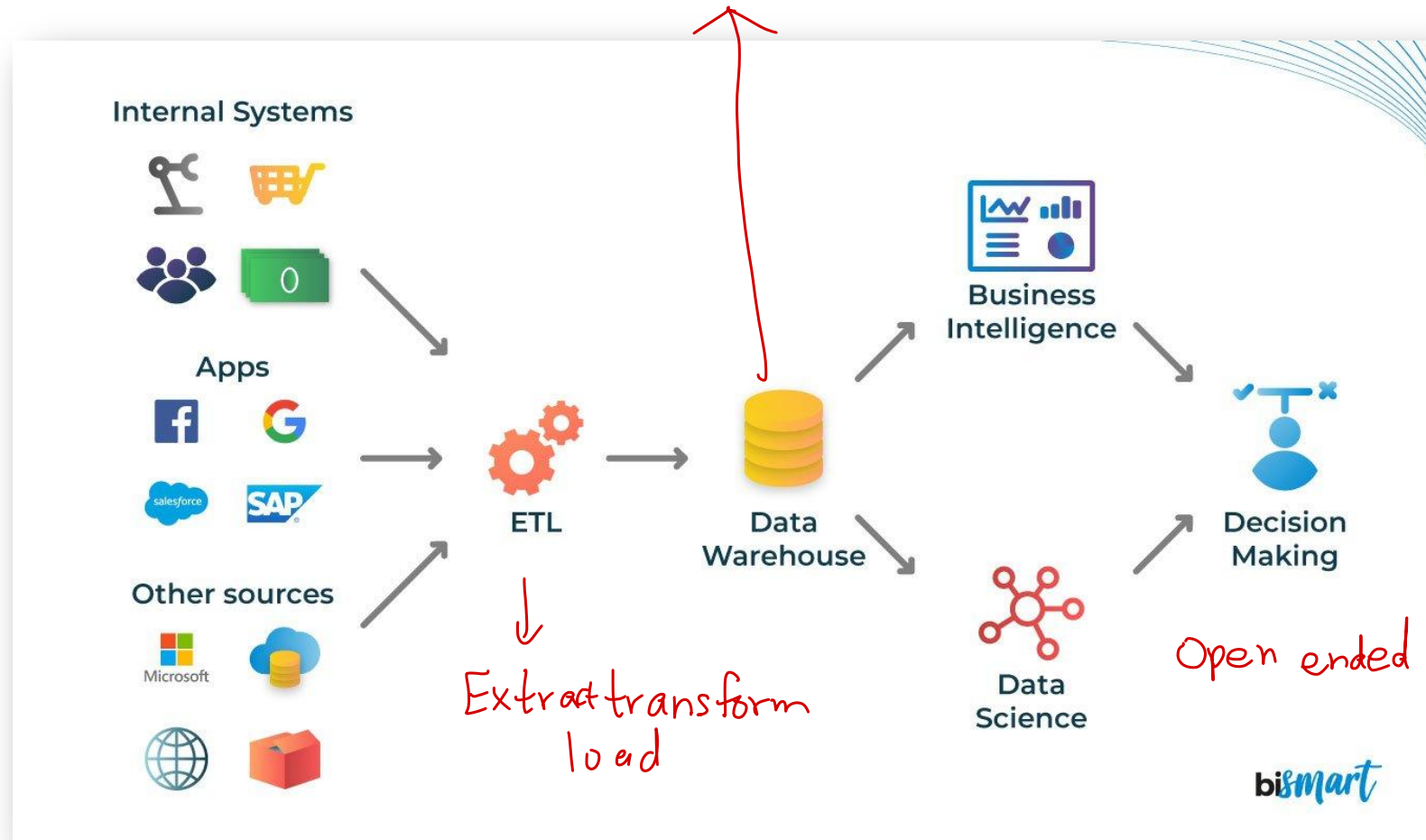
Requirements

- Immutable records (no edits)
- Full history tracking
- Verifiability / auditability



Data Integration

A **data warehouse** functions as the central data repository for an organization, acting as a convergence point for information from various sources. This process involves the extraction of data from its original sources, followed by integration into the data warehouse.



Data Warehouse vs. Data Lake

Both data warehouses and data lakes hold data for a variety of needs. The primary difference is that a data lake holds raw data of which the goal has not yet been determined. A data warehouse, on the other hand, holds refined data that has been filtered to be used for a specific purpose.

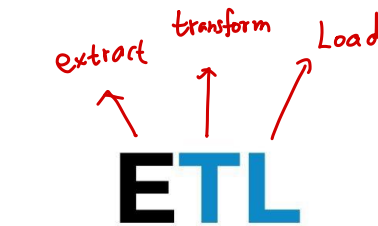
Data lakes are primarily used by data scientists while data warehouses are most often used by business professionals. Data lakes are also more easily accessible and easier to update while data warehouses are more structured and any changes are more costly.

— not always the case

[Source](#)

Building an ETL Data Pipeline with Python and SQL

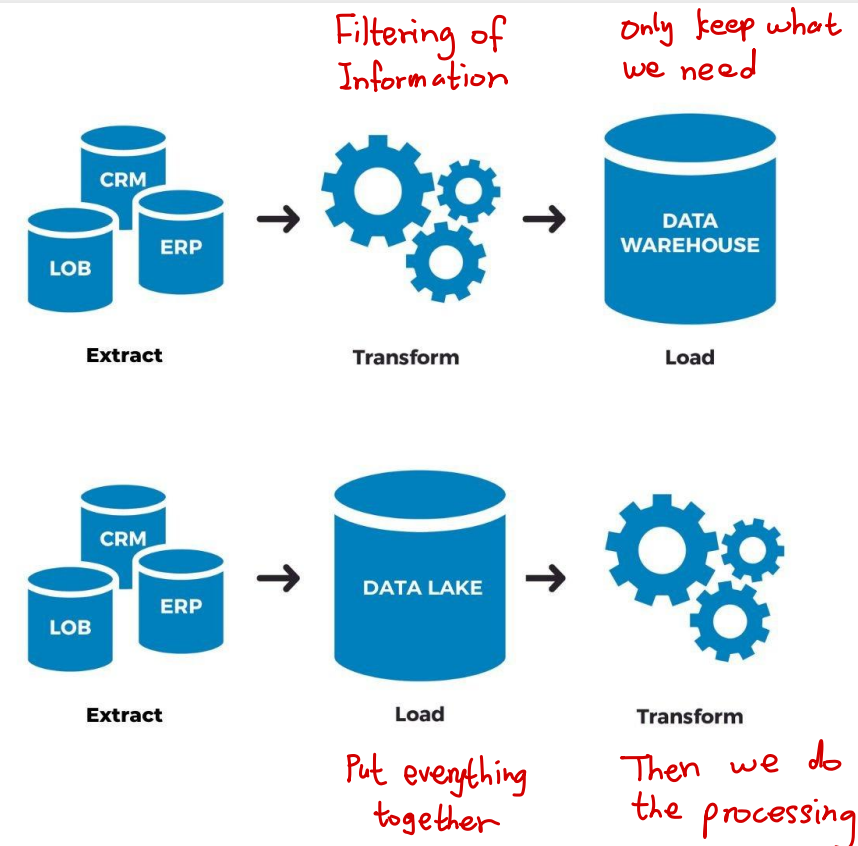
 Panagiotis Chatzinikolaou · [Follow](#)
Published in Level Up Coding · 8 min read · Oct 15, 2024

[Source](#)

VS.

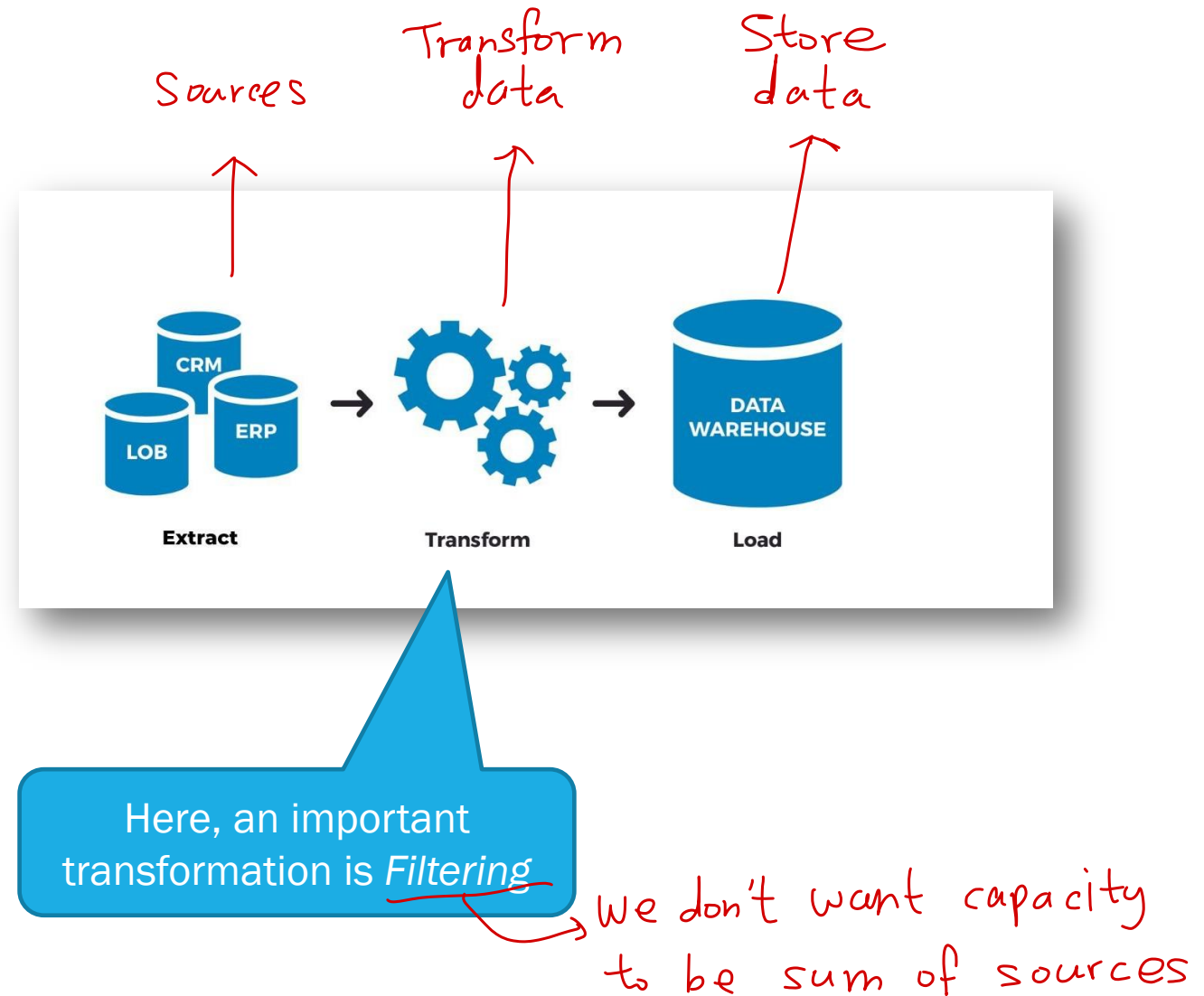
ELT

bismart



Extract, transform, load (ETL) process

extract, transform, load (ETL) is a data pipeline used to collect data from various sources. It then transforms the data according to business rules, and it loads the data into a destination data store.

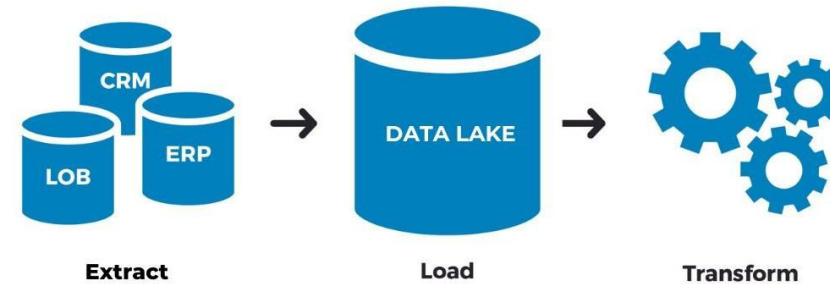


Extract, load, transform (ELT)

Extract, load, transform (ELT) differs from ETL in where the transformation takes place.

Instead of using a separate transformation engine, the processing capabilities of the target data store are used to transform data. This simplifies the architecture by removing the transformation engine from the pipeline.

ELT only works well when the target system is powerful enough to transform the data efficiently.



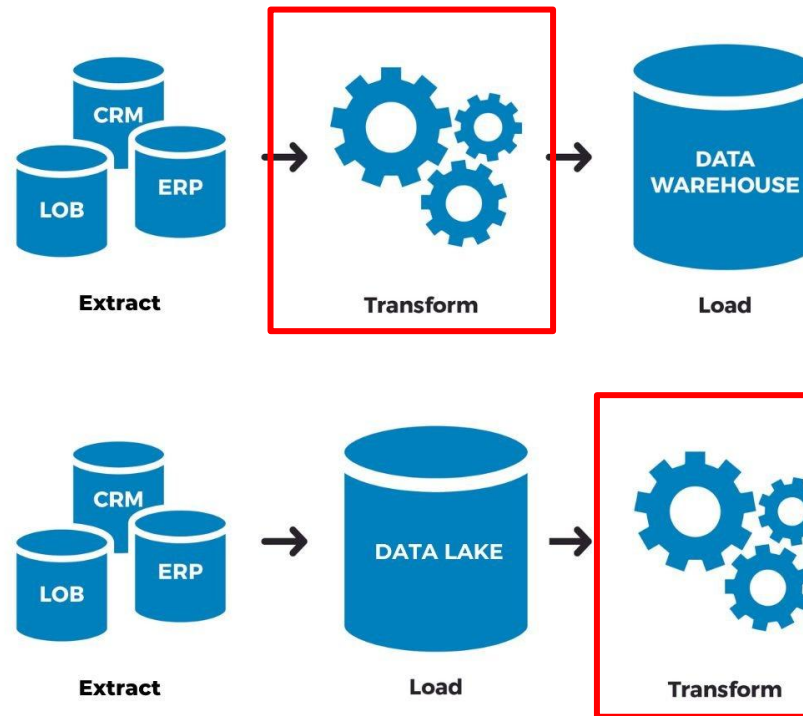
Favored approach for analytics

ETL

VS.

ELT

bi&mart



What are these transformations?

Important

Before or after



Data Transformation in ETL

[Source](#)

Types of transformations

1. Deduplication
2. Derivation
3. Aggregating
4. Splitting
5. Cleaning
6. Sorting
7. Filtering
8. Joining
9. Mapping

not covered

→ More often used in databases

We have already explored many of these

Understanding Data Transformation in ETL (Extract, Transform, Load)



Dr. Sanjay Kumar

Follow

3 min read · Jun 3, 2025

[Source](#)

ETL Transformation Techniques

Characteristic	Description	Example
Data Cleaning	Fixes errors and inconsistencies	Replace missing "Country" values with "Unknown"
Data Mapping	Aligns source fields to target schema	Map "cust_id" → "customer_id"
Aggregation	Summarizes data to higher levels	Convert daily sales to monthly totals
Bucketing/Binning	Groups numeric data into ranges	Group customer ages into 18-25, 26-35, etc.
Data Derivation	Creates new fields with logic	"Profit = Revenue - Cost"
Filtering	Selects relevant data only	Keep only sales from the year 2024
Joining	Combines multiple datasets	Merge "Customers" and "Orders" on "CustomerID"
Splitting	Breaks fields into components	Split "Full Name" into "First" and "Last"
Normalization	Unifies scales or units	Convert all prices to USD
Sorting/Ordering	Organizes records for usability	Sort transactions by "Signup Date"

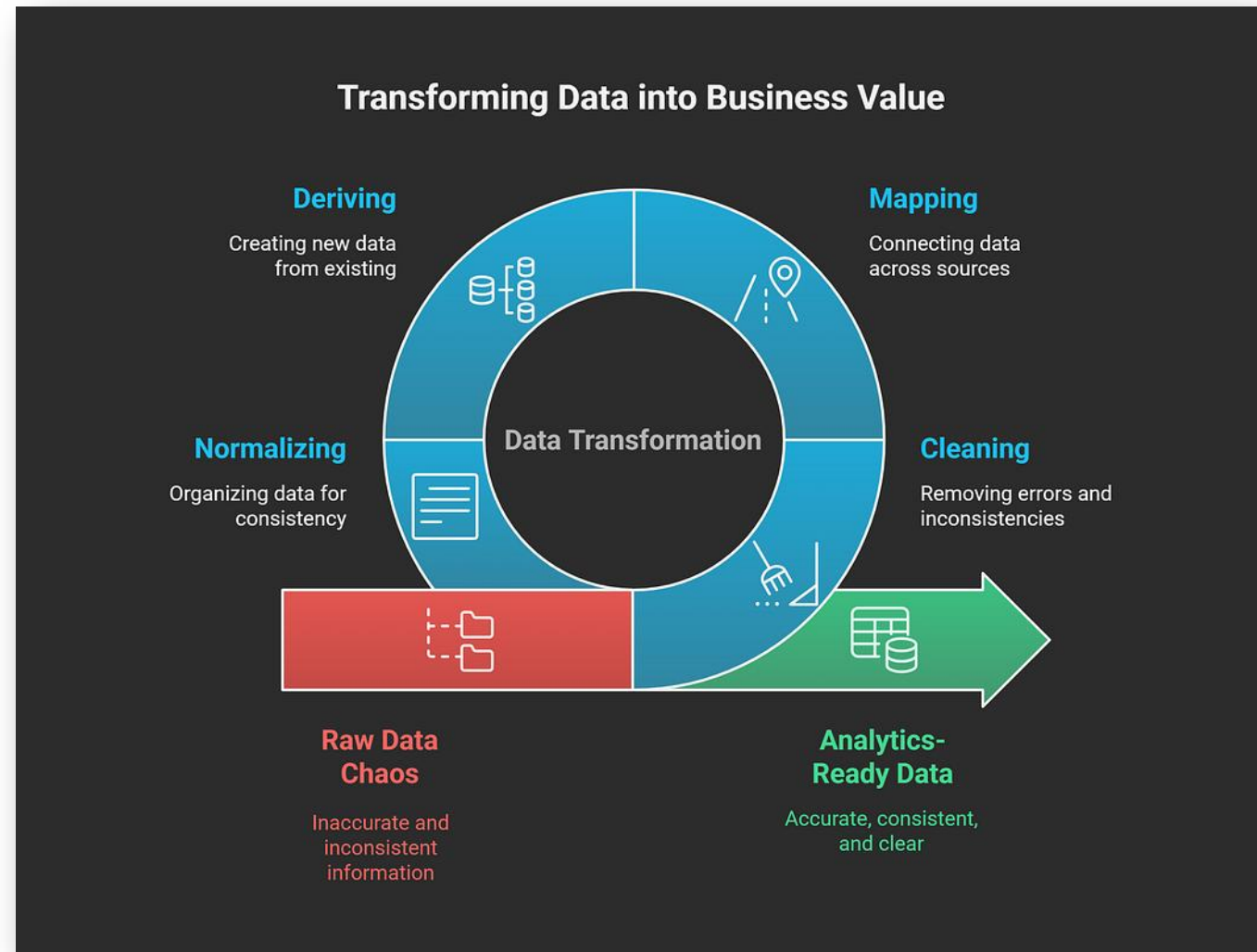
Understanding Data Transformation in ETL (Extract, Transform, Load)

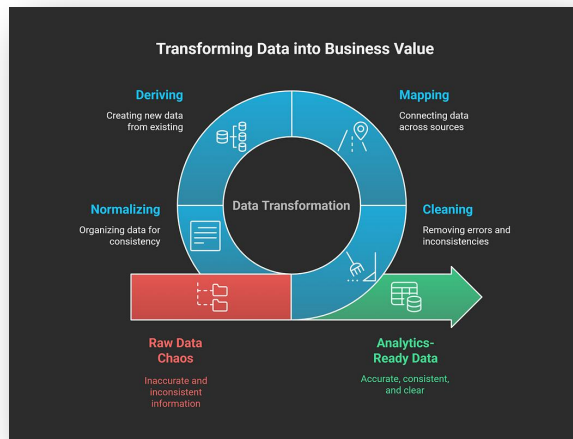
Dr. Sanjay Kumar [Follow](#) 3 min read · Jun 3, 2025

[Source](#)

Use two running examples

- (1) medical domain
- (2) urban planning





Data Cleaning

Clean the Data

Deduplication
Validity Checks
Outlier Detection
Missing Data

Deduplication

Before (Duplicates present)

patient_id	name	date_of_birth	visit_date	diagnosis
P001	John Smith	1980-05-12	2024-01-10	Hypertension
P001	John Smith	1980-05-12	2024-01-10	Hypertension
P002	Maria Lopez	1992-07-03	2024-02-15	Diabetes
P003	Ahmed Khan	1975-11-20	2024-03-01	Asthma
P002	Maria Lopez	1992-07-03	2024-02-15	Diabetes

After (Duplicates removed)

patient_id	name	date_of_birth	visit_date	diagnosis
P001	John Smith	1980-05-12	2024-01-10	Hypertension
P002	Maria Lopez	1992-07-03	2024-02-15	Diabetes
P003	Ahmed Khan	1975-11-20	2024-03-01	Asthma



Remember the near-duplicates... this would often happen when joining

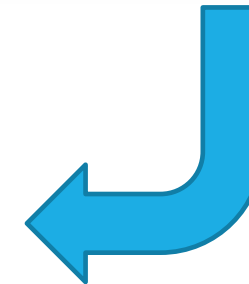
near-duplicates

Before (Near-duplicates)

patient_id	name	date_of_birth	visit_date	diagnosis
P001	John Smith	1980-05-12	2024-01-10	Hypertension
P009	J. Smith	1980-05-12	2024-01-10	Hypertension
P002	Maria Lopez	1992-07-03	2024-02-15	Diabetes
P010	Maria L. Lopez	1992-07-03	2024-02-15	Diabetes
P003	Ahmed Khan	1975-11-20	2024-03-01	Asthma

*Entity
resolution*

patient_id	name	date_of_birth	visit_date	diagnosis
P001	John Smith	1980-05-12	2024-01-10	Hypertension
P002	Maria Lopez	1992-07-03	2024-02-15	Diabetes
P003	Ahmed Khan	1975-11-20	2024-03-01	Asthma



Think of your decision as a claim
for which you need to bring
supporting evidence



Claim

- These two X (visits, persons, etc) are the same.

Type of evidence:

- Exact match on some key fields
- Numeric tolerance on numerical fields
- String similarity using known measures (edit distance, jaccard)

Validity Check

Before (Invalid data present)

patient_id	name	date_of_birth	visit_date	diagnosis	age
P001	John Smith	1980-05-12	2024-01-10	Hypertension	43
P002	Maria Lopez	1992/07/03	2024-02-15	Diabetes	-5
P003	Ahmed Khan	1975-11-20	2030-03-01	Asthma	49
P004	Li Wei	1988-13-40	2024-03-05	Flu	36
P005	Sara O'Neil	1990-06-25	2024-02-30	Cold	34

After (Cleaned / validated)

patient_id	name	date_of_birth	visit_date	diagnosis	age
P001	John Smith	1980-05-12	2024-01-10	Hypertension	43
P002	Maria Lopez	1992-07-03	2024-02-15	Diabetes	NULL
P003	Ahmed Khan	1975-11-20	NULL	Asthma	49
P004	Li Wei	NULL	2024-03-05	Flu	36
P005	Sara O'Neil	1990-06-25	NULL	Cold	34





Claim

- This value X is impossible (outside of range)
- This value X is not written in the proper format.
- This value X does not follow the appropriate standard.

Type of evidence:

- Domains for different attributes (e.g. age, salary)
- Knowledge of standards
- Knowledge of formats
- Temporal reasoning (A cannot happen before B)

Outlier Handling

Before (Outliers present)

patient_id	name	age	systolic_bp
P001	John Smith	43	120
P002	Maria Lopez	51	135
P003	Ahmed Khan	47	128
P004	Li Wei	45	250
P005	Sara O'Neil	50	132

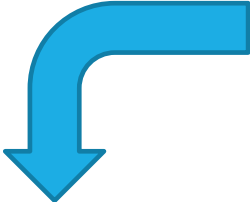
Option 1

patient_id	name	age	systolic_bp
P001	John Smith	43	120
P002	Maria Lopez	51	135
P003	Ahmed Khan	47	128
P004	Li Wei	45	180
P005	Sara O'Neil	50	132



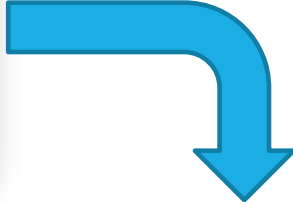
Domain knowledge
required

patient_id	name	age	systolic_bp
P001	John Smith	43	120
P002	Maria Lopez	51	135
P003	Ahmed Khan	47	128
P004	Li Wei	45	250
P005	Sara O'Neil	50	132



Option 2

patient_id	name	age	systolic_bp
P001	John Smith	43	120
P002	Maria Lopez	51	135
P003	Ahmed Khan	47	128
P005	Sara O'Neil	50	132



Option 3

patient_id	name	age	systolic_bp	is_outlier
P001	John Smith	43	120	0
P002	Maria Lopez	51	135	0
P003	Ahmed Khan	47	128	0
P004	Li Wei	45	250	1
P005	Sara O'Neil	50	132	0



Flag for review

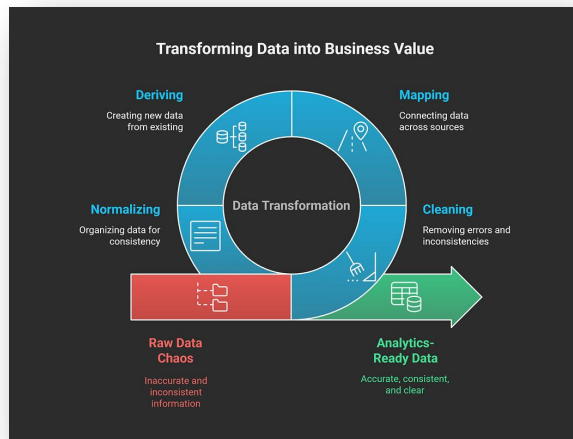
Missing Data

Before (Missing values)

patient_id	name	age	weight_kg	diagnosis
P001	John Smith	43	70	Hypertension
P002	Maria Lopez	NULL	68	Diabetes
P003	Ahmed Khan	47	NULL	Hypertension
P004	Li Wei	45	60	NULL
P005	Sara O'Neil	NULL	NULL	Hypertension

patient_id	name	age	weight_kg	diagnosis
P001	John Smith	43	70	Hypertension
P002	Maria Lopez	45	68	Diabetes
P003	Ahmed Khan	47	66	Hypertension
P004	Li Wei	45	60	Unknown
P005	Sara O'Neil	45	66	Hypertension





Standardize Formats

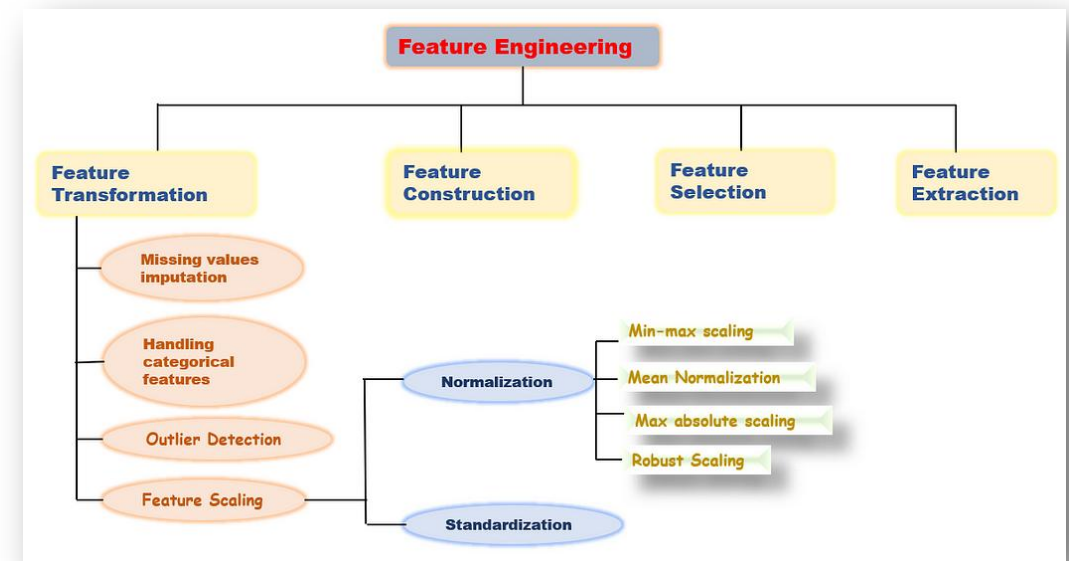
Data Normalization

Normalization/Standardization = Making data **uniform** in representation without changing its meaning

Look into:

- Uniform casing for text representation
- Code checking (using standard codes)
- Unit normalization (all same units)

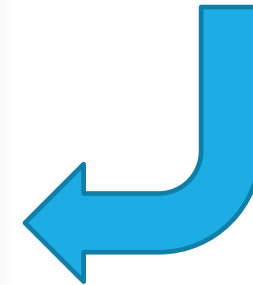
NOT the same meaning



patient_id	name	gender	weight	height	diagnosis
P001	john smith	M	70 kg	175 cm	hypertension
P002	Maria Lopez	Female	154 lb	1.65 m	Hypertension
P003	AHMED KHAN	male	80 KG	170cm	HYPERTENSION
P004	Li Wei	F	60kg	165 CM	diabetes

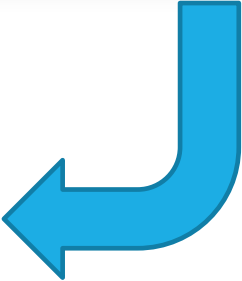
After (Standardized)

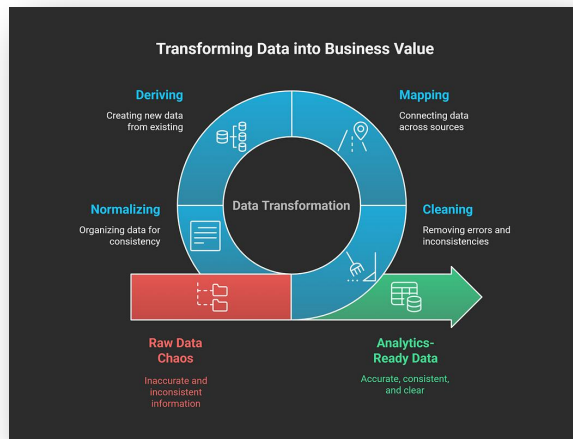
patient_id	name	gender	weight_kg	height_cm	diagnosis
P001	John Smith	Male	70	175	Hypertension
P002	Maria Lopez	Female	70	165	Hypertension
P003	Ahmed Khan	Male	80	170	Hypertension
P004	Li Wei	Female	60	165	Diabetes



sensor_id	location	timestamp	vehicle_count	speed
S101	Main & 1st Ave	04/01/2024 08:00 AM	120	45 km/h
S102	main and 1st avenue	2024-04-01 08:00	120	28 mph
S103	Pine & 3rd Ave	2024/04/01 08:05	95	50 kmh
S104	PINE & THIRD AVE	April 1, 2024 08:05	95	50 KM/H

After (Standardized)				
sensor_id	location	timestamp	vehicle_count	speed_kmh
S101	Main & 1st Ave	2024-04-01 08:00	120	45
S102	Main & 1st Ave	2024-04-01 08:00	120	45
S103	Pine & 3rd Ave	2024-04-01 08:05	95	50
S104	Pine & 3rd Ave	2024-04-01 08:05	95	50

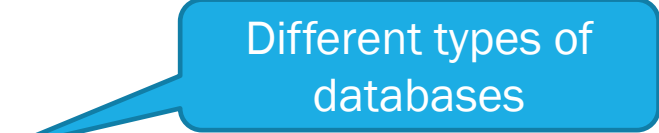




Reshape or Re-map 

Data Mapping *Mismatches*

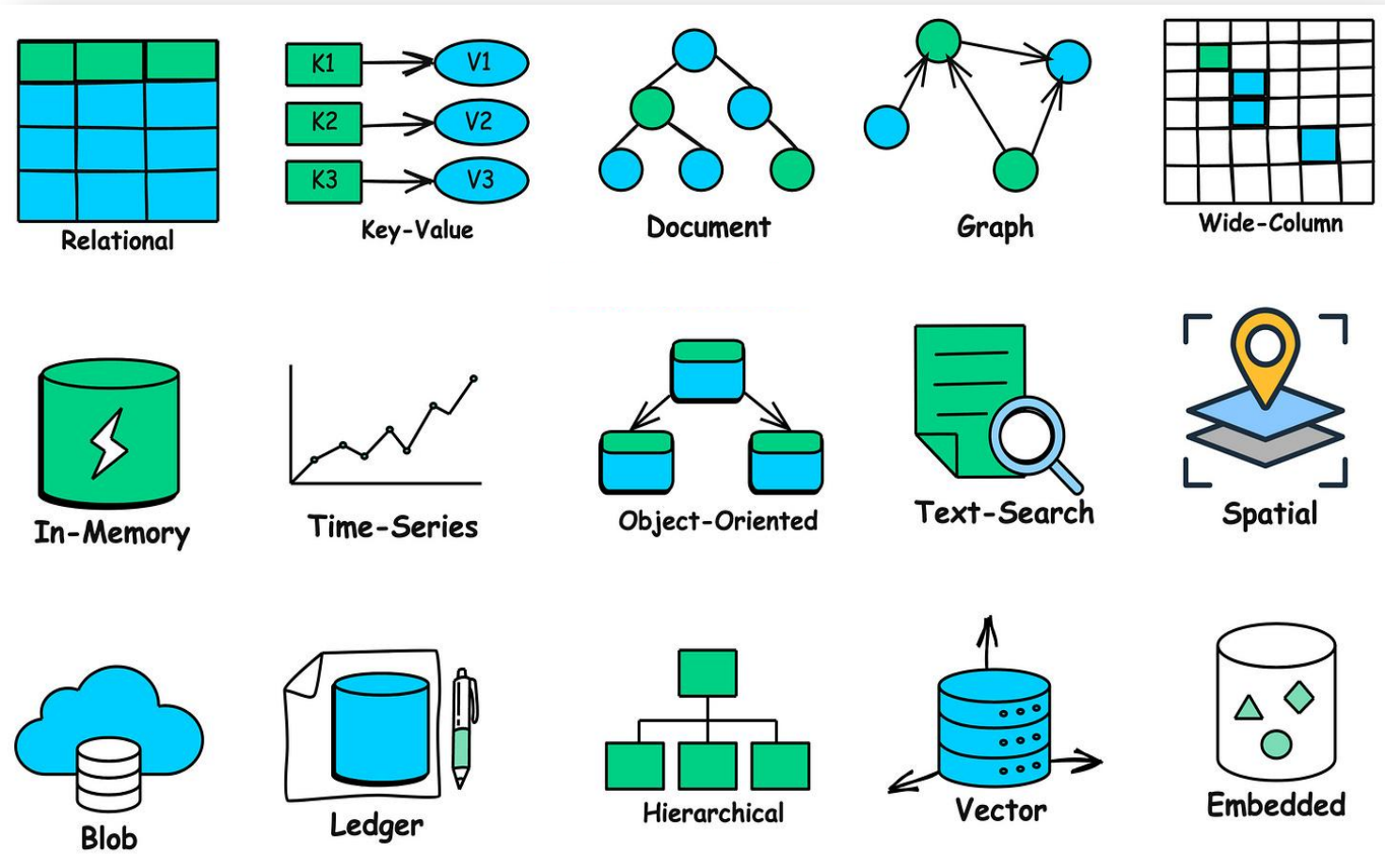
Mapping mismatches

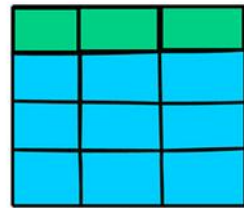


Different types of
databases

1. Schema mismatch
2. Feature name mismatch
3. Identifier mismatch
4. Unit mismatch
5. Data granularity mismatch

Schema mismatch

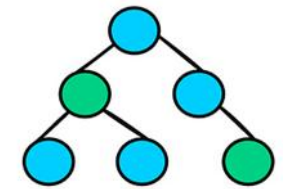




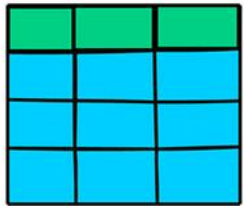
Relational

patient_id	name	condition	medication
101	Alice	Diabetes	Metformin
101	Alice	Hypertension	Lisinopril
102	Bob	Asthma	Albuterol

```
{  
  "patient_id": 101,  
  "name": "Alice",  
  "medical_history": [  
    { "condition": "Diabetes", "medications": ["Metformin"] },  
    { "condition": "Hypertension", "medications": ["Lisinopril"] }  
  ]  
}
```



Document



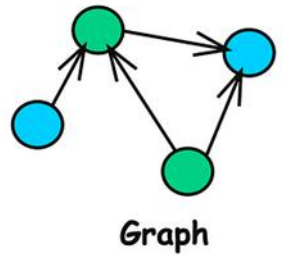
Relational



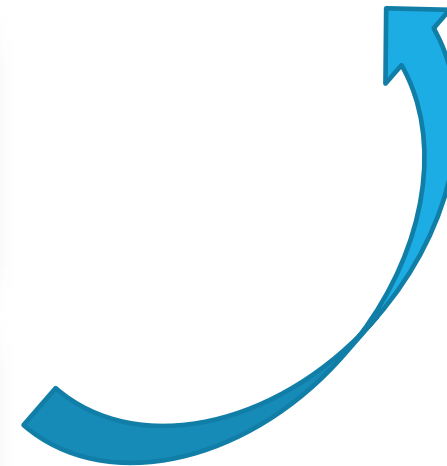
```
(Alice Kim) -[:TREATED_BY]-> (Dr. Smith)
(John Doe) -[:TREATED_BY]-> (Dr. Adams)
(Dr. Smith) -[:SPECIALIZES_IN]-> (Cardiology)
(Dr. Adams) -[:SPECIALIZES_IN]-> (Dermatology)
```

Patient_ID	Name	Doctor_ID
101	Alice Kim	9001
102	John Doe	9002

Doctor_ID	Doctor_Name	Specialty
9001	Dr. Smith	Cardiology
9002	Dr. Adams	Dermatology



Graph



Feature Name Mismatch

Source 1 (Hospital Database)

Patient_ID	Name	DOB
HOSP-101	Alice Kim	1985-07-12
HOSP-102	John Doe	1992-11-30

Source 2 (Insurance Database)

Insurance_ID	Full_Name	Birth_Date
INS-200A	Alice Kim	12-July-85
INS-300B	John Doe	30-Nov-92



The 6 data quality dimensions with examples

1. Completeness

How complete is your dataset?

2. Accuracy

Is the data an accurate representation of the element it describes?

3. Consistency

Is your data synchronized across your organization?

4. Validity

Does the data fit the defined range and definition?

5. Uniqueness

Are there any duplicates within your data?

6. Integrity

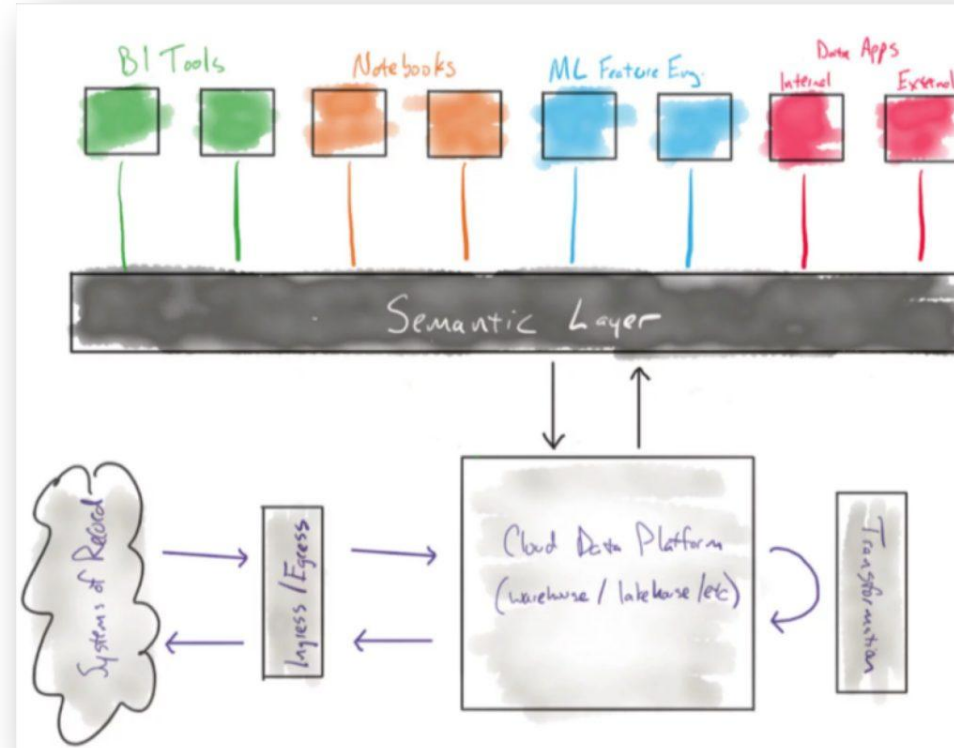
Can your data be traced and connected across the organization?

3. Consistency

Is your data
synchronized across
your organization?



Many datasets must be « coordinated » for consistency

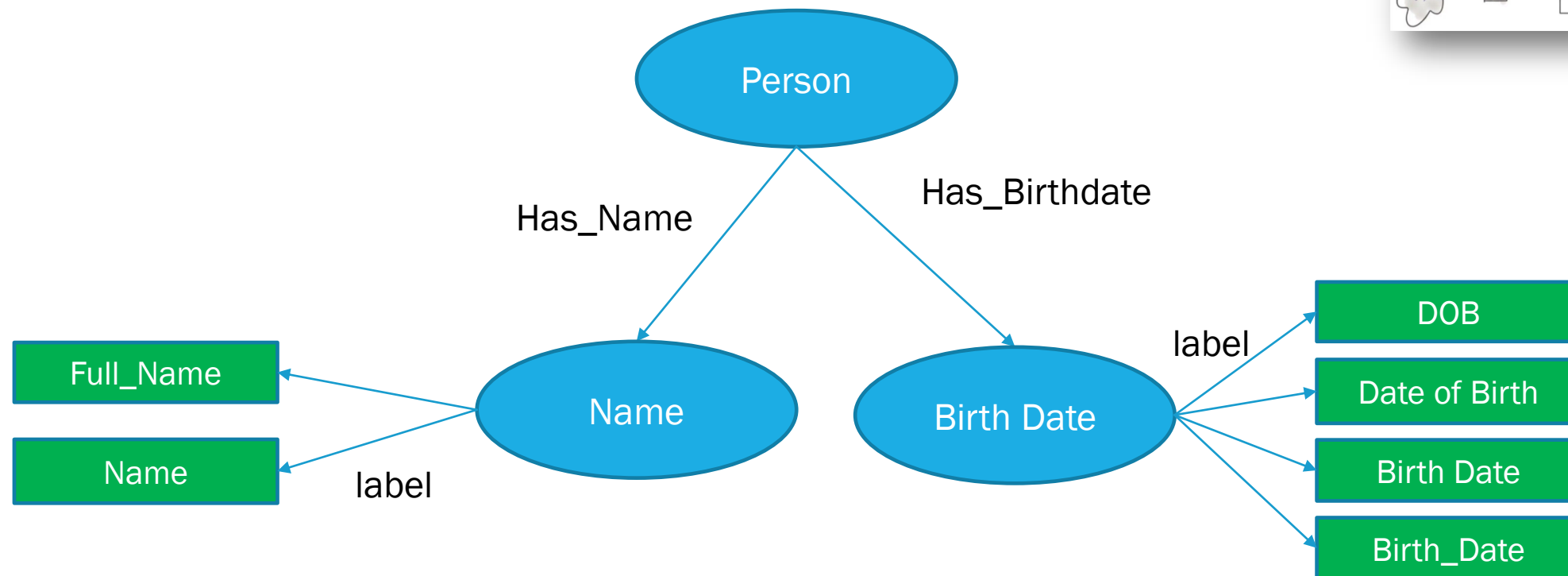
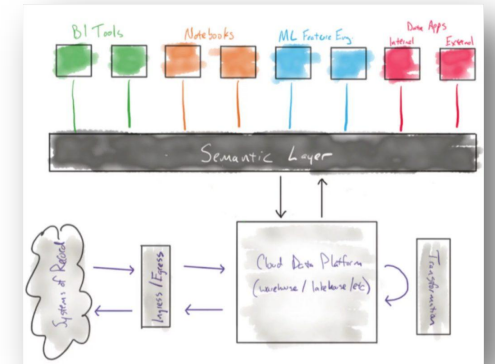


Column names (surface form)
How-Much, Cost, Price, Amount

How much money is paid (meaning)

Source

Semantic Level



Identifier Mismatch

Source 1 (Hospital Database)

Patient_ID	Name	DOB
HOSP-101	Alice Kim	1985-07-12
HOSP-102	John Doe	1992-11-30

Source 2 (Insurance Database)

Insurance_ID	Full_Name	Birth_Date
INS-200A	Alice Kim	12-July-85
INS-300B	John Doe	30-Nov-92

Which problem is that similar to?

near-duplicates

Unit Mismatch

Database A (U.S. Standard - Pounds)

Patient_ID	Name	Weight (lbs)	Date
201	Emma Li	150	2024-03-25
202	Jack Chen	175	2024-03-25
204	Noah Brown	160	2024-03-25

Database B (International Standard - Kilograms)

Patient_ID	Name	Weight (kg)	Date
201	Emma Li	68.0	2024-03-25
203	Mia Singh	59.9	2024-03-25
205	Liam Patel	82.0	2024-03-25

Requires knowledge of
conversion rules (often part
of « integrating solutions »)

Data Granularity Mismatch

Doctor_ID	Number_of_Patients
9001	3
9002	2

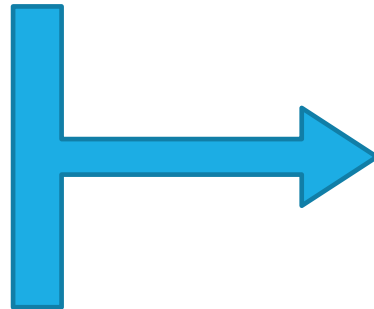
coarse grained

```
(Patient_A) -[:TREATED_BY]-> (Doctor_9001)
(Patient_B) -[:TREATED_BY]-> (Doctor_9001)
(Patient_C) -[:TREATED_BY]-> (Doctor_9001)
(Patient_D) -[:TREATED_BY]-> (Doctor_9002)
(Patient_E) -[:TREATED_BY]-> (Doctor_9002)
```

Fine grained

Time series 1 – Heart rate with 1 second intervals

Timestamp	Patient_ID	Heart_Rate (bpm)
2024-03-31 08:00	101	75
2024-03-31 08:01	101	76
2024-03-31 08:02	101	78



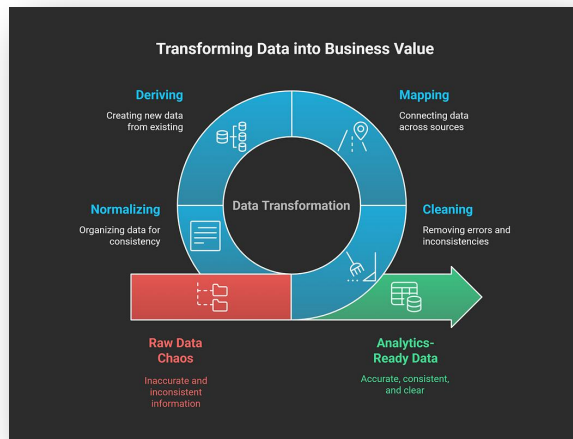
Target (Unified Time-Series Data with Interpolation)

Timestamp	Patient_ID	Heart_Rate (bpm)	Systolic	Diastolic
2024-03-31 08:00	101	75	120	80
2024-03-31 08:01	101	76	120	80
2024-03-31 08:02	101	78	120	80
2024-03-31 08:03	101	77	120	80
2024-03-31 08:04	101	76	120	80
2024-03-31 08:05	101	75	125	82

Time series 2 – Blood pressure with 5 second intervals

Timestamp	Patient_ID	Systolic	Diastolic
2024-03-31 08:00	101	120	80
2024-03-31 08:05	101	125	82

Interpolate what is missing



Data Mapping *Conflicts*

patient_id	dob	source
P001	1980-05-12	Hospital
P001	1980-05-13	Insurance
P001	1980-05-12	Lab system

Voting?

Authoritative
Source?

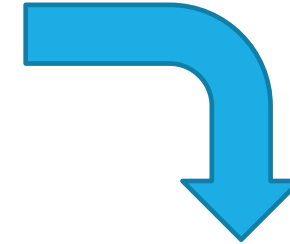
Flag for manual
review?

Resolution approaches

1. Authoritative Source
2. Consensus / Voting
3. Statistical Reconciliation
4. Confidence-based weighting
5. Flag for Manual Review

1. Authoritative Source

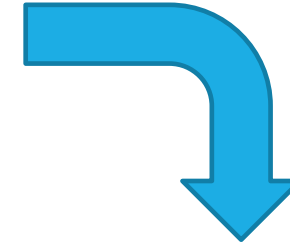
patient_id	dob	source
P001	1980-05-12	Hospital
P001	1980-05-13	Insurance



patient_id	dob
P001	1980-05-12

2. Consensus / Voting

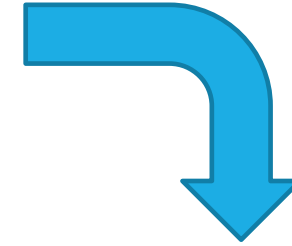
zone_id	land_use	source
Z001	Residential	City DB
Z001	Mixed-use	Urban survey
Z001	Residential	Satellite map



zone_id	land_use
Z001	Residential

3. Statistical Reconciliation

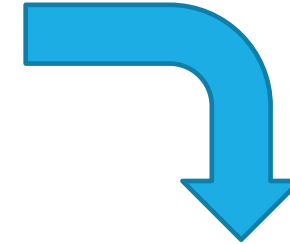
patient_id	glucose_mg_dl	source
P003	104	Lab A
P003	108	Lab B
P003	102	Lab C



patient_id	glucose_mg_dl
P003	104.7

4. Confidence Based Weighting

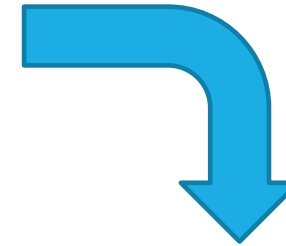
sensor_id	avg_speed_kmh	source	reliability
S101	70	City sensor	0.8
S101	65	Traffic API	0.5



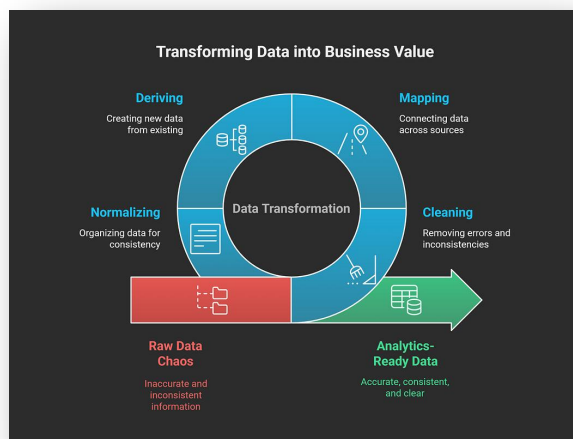
sensor_id	avg_speed_kmh
S101	68.2

5. Flag for Manual Review

patient_id	address	source
P004	123 Maple St	Hospital
P004	125 Maple St	Lab

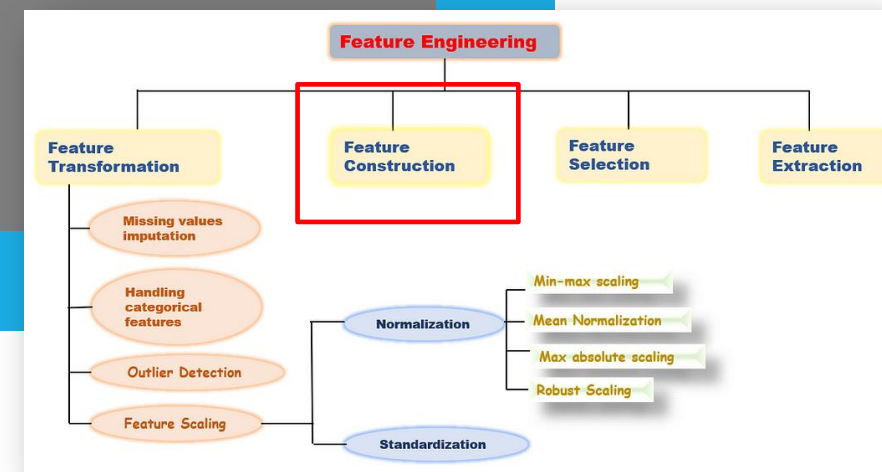


patient_id	address	resolution_method
P004	[CONFLICT]	Flag for Manual Review



Enrich with Derived Fields

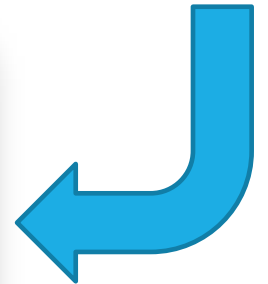
Data Enrichment



Derivation

road_id	vehicle_count	time_window_min	lane_count
R1	200	10	2
R2	150	10	3
R3	400	20	2
R4	180	10	1

road_id	vehicle_count	time_window_min	lane_count	vehicles_per_min	density_per_lane	congestion_level
R1	200	10	2	20.0	10.0	Medium
R2	150	10	3	15.0	5.0	Low
R3	400	20	2	20.0	10.0	Medium
R4	180	10	1	18.0	18.0	High



Aggregation

Example: Hourly traffic flow

hour	total_vehicles	avg_speed
08:00	450	42
09:00	520	40
10:00	380	45

Example: Daily vehicle count per sensor

sensor_id	date	total_vehicles
S101	2024-04-01	1200
S102	2024-04-01	950
S103	2024-04-01	1100

Time-based
aggregation

Example: Average traffic per street

street	avg_vehicle_count	avg_speed
Main Street	120	45
Pine Avenue	95	50
Oak Boulevard	110	48

Example: Total vehicles per neighborhood

neighborhood	total_vehicles
Downtown	4500
Midtown	3800
Riverside	3200

Spatial-based
aggregation

Splitting

sensor_id	location
S101	Main & 1st Ave
S102	Pine & 3rd Ave

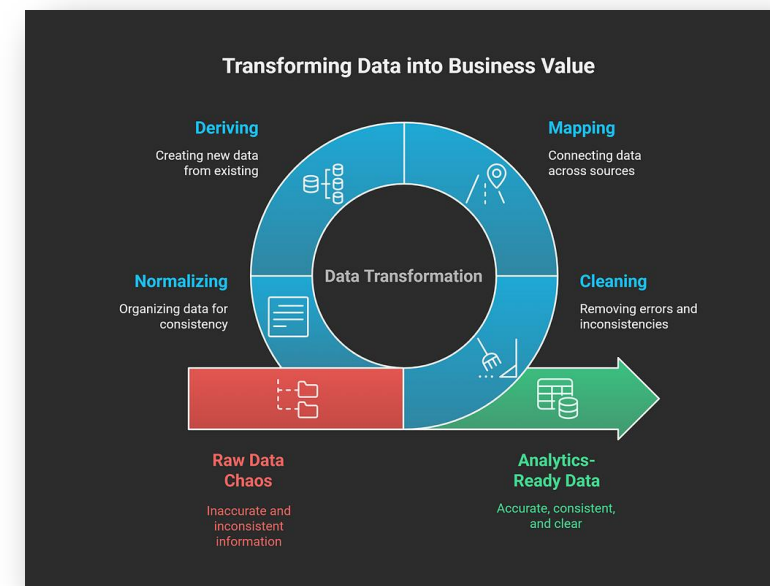
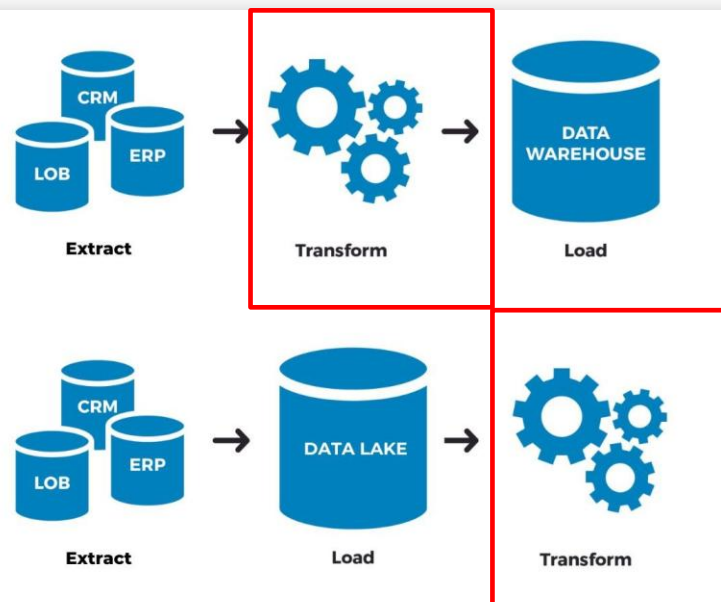
Split into: `street1` and `street2`

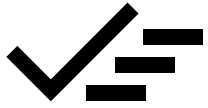
sensor_id	street1	street2
S101	Main	1st Ave
S102	Pine	3rd Ave



ETL VS. ELT

bi&mart





DATA INTEGRATION

- Case study (Database review)
- Data Integration – ETL/ELT
- Transformation
 - Cleaning
 - Normalizing
 - Mapping
 - Deriving